

README

Part 6: Transformer / GPT — 从零构建一个迷你 ChatGPT

> 200 行代码，训练一个 decoder-only Transformer，让神经网络写出"莎士比亚"。

章节导航

	内容
[Data Tokenizer](01_data_and_tokenizer.md)	ChatGPT 动机、tiny Shakespeare、字符级 tokenizer、train/val 划分、Dataloader、Bigram 基线、交叉熵、Adam
[Attention 从零开始](02_attention_from_scratch.md)	聚合的数学技巧三版本、位置编码、self-attention 单头 (K/Q/V)、6 条 attention 笔记、Multi-Head
[Transformer Block](03_transformer_block.md)	FeedForward、残差连接、LayerNorm (pre-norm)、完整 decoder-only Transformer、Dropout、Scale Up 与生
[Beyond Transformer](04_beyond_transformer.md)	Encoder vs Decoder、Cross-Attention、nanoGPT 走读、回到 ChatGPT/GPT-3、预训练 vs 微调、RLHF

前置知识

本部分需要你已掌握：

- Python + PyTorch 基础：nn.Module、nn.Embedding、F.softmax、交叉熵损失
- 语言建模框架：Part 1/2 里"预测下一个 token、负对数似然损失、训练循环"的整套思路
- BatchNorm 的训练/推理两态：Part 3 的 02_batchnorm.md —— 讲 LayerNorm 时会和它对照
- 手动反向传播的直觉：Part 4 的 02_forward_and_backward.md ("加法节点把梯度均分给两个分支") —— 讲残差连接时会用到
- 卷积的空间性：Part 5 的 03_training_and_bugs.md (WaveNet 的感受野 / 空间性) —— 讲 attention"无空间概念"时会和卷积对比

> 如果你卡住了，随时回看前几章的 tutorial/ 目录。

学习路线图

Part 5 (WaveNet / 卷积层次化融合)

|
| "卷积有空间性、有固定感受野，能不能让 token 自己决定看谁？"



Part 6: Transformer / GPT	
① 数据与 Tokenizer — 字符级编码	→ 01_data_and_tokenizer.md
② Attention 从零开始 — 通信机制	→ 02_attention_from_scratch.md
③ Transformer Block — 通信+计算+残差	→ 03_transformer_block.md
④ 超越 Transformer — GPT-3 / RLHF	→ 04_beyond_transformer.md

| "理解了 attention，下一步是让模型学会推理..."

Transformer 之后（阅读 Karpathy 的 micrograd / minGPT 等）

学完这一部分你能...

- 讲清楚 ChatGPT 底层就是语言模型，以及 GPT = Generative Pretrained Transformer
- 手写字符级 tokenizer (encode/decode)，理解字符级 vs subword (BPE/sentencepiece) 的取舍
- 实现 Dataloader：一个 chunk 里装多个样本、随机 offset 采样、batch 独立并行
- 理解 attention 的数学技巧：用 torch.tril + softmax 做加权聚合
- 从零实现 self-attention 单头 (query/key/value、亲和力、缩放、遮罩)，并展开 6 条 attention 笔记
- 组装 Multi-Head + FeedForward + 残差连接 + LayerNorm (pre-norm) 的完整 Transformer Block
- 用 Dropout 正则化、理解 scale up 后 loss 如何从 2.5 一路降到 1.48
- 区分 encoder / decoder / 完整架构，看懂 nanoGPT 代码
- 讲清楚 ChatGPT 的预训练（文档补全器）→ 微调（SFT → 奖励模型 → RLHF）全流程

演进路线：loss 一路怎么降的

本教程会反复看到这张表。两套数字：

阶段	Karpathy 视频	本仓库脚本 (CPU 小规模)
Bigram 初始	~4.87	~4.74
Bigram 训练后	≈ 2.5	≈ 2.50
+ 单头 self-attention	2.4	≈ 2.39
+ multi-head	2.28	≈ 2.45
+ feedforward	2.24	≈ 2.50 †
+ 残差连接	2.08	≈ 2.23
+ LayerNorm	2.06	≈ 2.23
Scale up (GPU)	1.48	CPU 缩小型 ≈ 2.80

> △ 视频里的数字是 A100 GPU + 完整超参跑出来的；我们的脚本是 CPU 缩小版（更小的 batch/block/层数/步数）。不同超参、不同随机种子，数字都会有差异，所以都带 ≈。看趋势，别死记数字。

>

> † Phase 2 (+feedforward) 在少步数下收敛偏慢、val loss 暂时高于 Phase 1 (+multi-head)，多跑步数会降下来（原视频 2.28 → 2.24）。详见 03_transformer_block.md 的解释。

课后作业

每一章末尾有 2-3 道思考题（折叠答案）。全部学完后，去这里做动手练习：

[Assignment 6](../assignments/assignment_6/)

相关资源

- Andrej Karpathy 原视频：[Let's build GPT: from scratch, in code, spelled out](https://www.youtube.com/watch?v=kCc8FmEb1nY)（makemore Part 6）
- Vaswani et al. 2017 论文：[Attention is All You Need](https://arxiv.org/abs/1706.03762)
- He et al. 2015：[Deep Residual Learning for Image Recognition](https://arxiv.org/abs/1512.03385)
- Srivastava et al. 2014：[Dropout: A Simple Way to Prevent Neural Networks from Overfitting](https://arxiv.org/abs/1207.0580)
- Ba et al. 2016：[Layer Normalization](https://arxiv.org/abs/1607.06450)
- [nanoGPT](https://github.com/karpathy/nanoGPT) — 训练 Transformer 的最简参考实现
- [GPT-3 论文](https://arxiv.org/abs/2005.14165)：175B 参数、300B tokens 预训练
- OpenAI 博客：[ChatGPT 对齐阶段（SFT → 奖励模型 → RLHF）](https://openai.com/blog/chatgpt)

[← 上一章：Part 5 WaveNet](../Part5_wavenet/tutorial/README.md)

01_data_and_tokenizer

01 — 数据与 Tokenizer：从 ChatGPT 到字符级语言模型

> 一切的起点：把莎士比亚文本变成神经网络能吃的整数序列，再喂给一个最简单的模型。

前置知识

本章需要你已掌握：

- Python + PyTorch 基础：nn.Module、nn.Embedding、F.softmax
 - 语言建模框架：Part 1/2 里"预测下一个 token、负对数似然损失、训练循环"的整套思路
- > Part 5 的 WaveNet 有助于理解本课后续的 attention 对比，但本章不依赖它。

从 Part 5 结束的地方出发

Part 5 我们用 WaveNet 把上下文层次化融合，在 names 数据集上把验证 loss 压到了 2.0 以下。但那套方案的骨架是卷积——卷积有固定的感受野、有空间性，你告诉网络"看这几个邻居"。

这一章我们换一个完全不同的思路：让 token 自己决定去看谁。这个思路就是 Transformer。

> 这不是推翻 Part 5，而是把它放到更大的图景里：WaveNet/CNN 是我们学习 Transformer 的"脚手架"。后面讲 attention "无空间概念"时，我们会专门拿卷积来做对比。

课程动机：ChatGPT 是什么？

你一定听说过 ChatGPT。它底层其实就是一个语言模型（language model）：给它一个序列的开头，它逐词（更准确说是逐 token）地"续写"下去。

你给："帮我写一首关于 AI 的俳句"

ChatGPT: "知识带来繁荣

拥抱它的力量

人类永向前" ← 从左到右，一次吐一个 token

同一个提示，两次回答不同——因为它是概率系统：每个位置都从"下一个 token 的概率分布"里采样。

- 所以 ChatGPT 做的事，本质上和我们前面 Part 1/2 做的一模一样：预测序列里的下一个 token。区别只在于规模、数据量和 token 的粒度。

GPT 三个字母拆开：

- Generative —— 生成式，能续写
- Pretrained —— 预训练，先在大量文本上训练
- Transformer —— 底层的神经网络架构

Transformer 的起源

Transformer 来自 2017 年那篇里程碑论文《Attention is All You Need》（Vaswani et al.）。有意思的是，这篇论文读起来像一篇随机的机器翻译论文——因为作者当时根本没预料到它会统治整个 AI 领域。它是在机器翻译的背景下提出来的，结果这个架构在之后 5 年里被"复制粘贴"进了 AI 的方方面面，包括 ChatGPT 的核心。

> Transformer：一种神经网络架构，核心组件是 attention（注意力）机制，让序列中的每个元素按"重要性"聚合其它元素的信息。

我们现在就动手训练一个字符级的 Transformer

语言模型。先定个小目标：让它能写出"看起来像莎士比亚"的文本。

数据：tiny Shakespeare

用互联网级别的数据当然不现实，我们用 Karpathy 最喜欢的小数据集——tiny Shakespeare：

文件：data/input.txt

内容：莎士比亚全部作品的拼接

大小：约 1.1 MB，约 100 万字符

[01_explore_data.py](../scripts/01_explore_data.py) 的统计输出（我们实跑的真实结果）：

—— 数据集统计 ——

总字符数: 1,115,394

前 1000 个字符长这样（开场是《科里奥兰纳斯》的平民戏）：

"First Citizen:\nBefore we proceed any further, hear me speak.\n\nAll:\nSpeak, speak.\n\nFirst Citizen:\nYou are all resolved rather to die than to famish?\n..."

> 我们做的是字符级语言模型：预测"下一个字符是什么"。ChatGPT 用的是子词级，后面我们会对比。

字符级 Tokenizer

"Tokenizer（分词器）"的意思是：把原始文本按某种词表转成整数序列。我们的词表就是"文本中出现过的所有字符"：

```
python
chars = sorted(list(set(text)))
vocab_size = len(chars)
```

set(text) 得到所有唯一字符，list 加排序得到一个稳定的顺序。运行结果：

```
——— 词汇表 ———
唯一字符数 (vocab_size): 65
字符列表:
!$&'-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
```

注意第一个字符（"）其实是换行符 \n，列表里依次是：空格、标点、数字（0-9，虽然莎士比亚里用得很少）、大写字母、小写字母——按 ASCII 排序。语言模型只能输出它见过的字符——这正是词汇表的作用。

encode / decode：字符 ⇌ 整数

构建两张查表字典，然后定义编码器和解码器：

```
python
stoi = {ch: i for i, ch in enumerate(chars)} # char → int
itos = {i: ch for i, ch in enumerate(chars)} # int → char
encode = lambda s: [stoi[c] for c in s] # 字符串 → 整数列表
decode = lambda l: ''.join([itos[i] for i in l]) # 整数列表 → 字符串
```

运行结果：

```
——— Tokenizer 演示 ———
encode('hi there') = [46, 47, 1, 58, 46, 43, 56, 43]
decode([46, 47, 1, 58, 46, 43, 56, 43]) = 'hi there'
往返一致: True
索引 0 对应字符: '\n'（换行符）
索引 1 对应字符: ''
```

- $\triangle$ 索引 0 通常是换行符 '\n'（注意它和空格 '' 是索引 1，两个不同字符！）。encode→decode 的"往返一致性"（decode(encode(s)) == s）是 tokenizer 正确性的基本自检。
- 用字符级的好处是简单：没有未知字符、不用训练 BPE 词表。代价是序列很长（一句话几十上百个整数）。这是本章的核心权衡，下面展开。

其它 Tokenizer 对比：词表大小 vs 序列长度

字符级只是众多 tokenizer 中的一种，而且是最简单的一种。工业界有更常见的方案：

Tokenizer	提出方	粒度	词表大小	说明
字符级	—	单字符	65（我们的）	最简单，序列最长
sentencepiece	Google	子词	几千~几万	实践中常见的 subword 方案
tiktoken / BPE	OpenAI	子词	~50,000（GPT-2）	GPT 系列用的字节对编码

用 OpenAI 的 tiktoken (BPE, GPT-2 词表) 编码 "hi there", 得到的不是 [46, 47, 1, ...], 而是只有 3 个整数, 每个整数的范围在 0 到 50,256 之间。

这就是核心权衡：

词表大 + 序列短 vs 词表小 + 序列长

(每个 token 信息量大) (每个 token 信息量小, 要更多步)

↓ ↓

50K tokens 的 BPE 65 tokens 的字符级

更贴近"词"的粒度 最简单、无训练开销

- subword (子词) tokenizer：既不是整词，也不是单字符，而是介于两者之间。BPE 从字符开始，逐步把高频相邻片段合并成新的 token，所以它能表达任何词、又能压缩序列长度。
- 我们这一课坚持用字符级，因为它是理解整套流程最干净脚手架。你把 encode/decode 换成任何别的 tokenizer，后面的训练代码一行都不用改。

训练 / 验证划分

把整个文本编码成一个大整数张量，然后划分：

```
python
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data))
train_data = data[:n]
val_data = data[n:]
```

运行结果：

—— Train/Val 划分 (90/10) ——

```
data shape: torch.Size([1115394]), dtype: torch.int64
train: 1,003,854 字符 (90.0%)
val: 111,540 字符 (10.0%)
```

- ⚠ 注意 dtype=torch.long——索引/标签必须用整数类型 (int64)，不能是 float，否则 nn.Embedding 和 F.cross_entropy 都会报错。
- 为什么要留 10% 的验证集？因为我们不想要一个"死记硬背"莎士比亚的模型，而想要一个能泛化、能"编造"莎士比亚风格文本的模型。验证集从头到尾不参与训练，用来检测过拟合。

DataLoader：不喂整篇文本，只采样 chunk

一个重要事实：我们永远不会把整篇 100 万字符喂给

Transformer，那在计算上不可行。训练时我们随机从数据里采小块 (chunk) 来训练。

两个关键超参数：

block_size：一块里放多少个字符 (= 上下文长度 / context length, 模型预测下一个字符时最多能看多远)
batch_size：每次并行处理多少个独立的块 (为了把 GPU 喂满)

一个 chunk 里藏着多个样本

关键洞察：一块 9 个字符的连续序列，其实包含了 8 个训练样本。因为每个位置都对应一个"用前 t 个字符预测第 t+1 个字符"的样本。[02_bigram_baseline.py](../scripts/02_bigram_baseline.py) 里我们用 `train_data[:block_size + 1]` 实际打印出来：

—— 一个 chunk 内含多个样本 ——

取前 $8+1=9$ 个字符，它们按顺序偏移形成 8 个训练样本：

```
x=[18] → y=47
x=[18, 47] → y=56
x=[18, 47, 56] → y=57
x=[18, 47, 56, 57] → y=58
x=[18, 47, 56, 57, 58] → y=1
x=[18, 47, 56, 57, 58, 1] → y=15
x=[18, 47, 56, 57, 58, 1, 15] → y=47
x=[18, 47, 56, 57, 58, 1, 15, 47] → y=58
```

- 这就是"一个 chunk 内含多个样本"：x 是前 `block_size` 个字符，y 是偏移一位的 `block_size` 个字符（T, T+1 偏移），两者都是 (B, T)。
- 训练时覆盖"上下文长度从 1 到 `block_size`"的所有情况，不只是为了计算效率——更重要的是让模型习惯各种长度的上下文。这样推理时从一个字符开始也能预测。

get_batch：随机 offset 采样 + torch.stack

```
python
def get_batch(split):
    data_local = train_data if split == 'train' else val_data
    ix = torch.randint(len(data_local) - block_size, (batch_size,))
    x = torch.stack([data_local[i:i + block_size] for i in ix]) # (B,T)
    y = torch.stack([data_local[i + 1:i + block_size + 1] for i in ix]) # (B,T)
    x, y = x.to(device), y.to(device)
    return x, y
```

随机采 `batch_size` 个起始位置 ix，每个位置切出 (i, i+block_size) 作为 x，(i+1, i+block_size+1) 作为 y，然后用 `torch.stack` 叠成一个 (B, T) 的 batch。运行结果：

—— 一个 batch ——

X shape: torch.Size([32, 8]) (B=32, T=8), Y shape: torch.Size([32, 8])
共 256 个独立样本打包在一个 batch 中

- ⚠ 为什么起始位置上限是 `len(data) - block_size`？因为要保证 `i+block_size` 不越界。
- batch 里的 32 个 chunk 是互相独立的——它们之间不通信。这是后面 attention 笔记 ③ 的伏笔（batch 维度上无通信）。

Bigram 语言模型：最弱的基线

在进入 Transformer 之前，先实现最简单的语言模型——Bigram（双字母模型）。Part 1/2
我们已经深入讲过它，这里快速过一遍。

它只有一张表：`nn.Embedding(vocab_size, vocab_size)`。输入一个 token 索引，直接查表得到"下一个 token 的分数 (logits)"。

```
python
```

```

class BigramLanguageModel(nn.Module):
    """最简单基线：每个 token 只看"我是谁"， token 之间不交流"""

    def __init__(self, vocab_size):
        super().__init__()
        self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)

    def forward(self, idx, targets=None):
        logits = self.token_embedding_table(idx) # (B,T,C)
        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B * T, C)
            targets = targets.view(B * T)
            loss = F.cross_entropy(logits, targets)
        return logits, loss

```

- $\triangle$ 预测时 token 完全不看上下文——只看"我是谁"。例如 token 5 只根据自己是 5 来预测下一个，因为某些字符后面常跟另一些字符，所以能学到一点点规律，但上下文信息全被浪费了。

交叉熵损失：为什么 reshape 成 (B*T, C)

交叉熵衡量"logits 对 targets 的预测质量"。我们想要：target 对应的那一维 logits 很高，其它维很低。

但 PyTorch 的 `F.cross_entropy` 对多维输入有形状要求：它希望 logits 是 (样本数, 类别数)。我们的 logits 是 (B, T, C)，所以要先合并 B 和 T 两个维度：

```

python
logits = logits.view(B * T, C) # (B, T, C) → (BT, C)，把"样本"摊平
targets = targets.view(B * T) # (B, T) → (BT,)
loss = F.cross_entropy(logits, targets)

```

- 这里的 C (channel) 就是 `vocab_size = 65`——交叉熵的"类别数"必须是词表大小。之所以要 reshape，是因为交叉熵把每一个 (batch, time) 位置都当成一个独立的分类样本。
- $\triangle$ `view` 只是改变张量的"视图"，不拷贝内存。-1 也能让 PyTorch 自动推断，但显式写 `B * T` 更清晰。

初始 loss 以及理论下限：

——— 初始 loss ———

初始 loss: 4.7417

理论下限: $-\ln(1/65) = \ln 65 \approx 4.17$ (完全均匀分布时的损失)

- 为什么理论下限是 4.17？如果模型完全随机、对 65 个字符一视同仁（每个概率 $1/65$ ），负对数似然就是 $-\ln(1/65) = \ln 65 \approx 4.17$ 。我们初始 4.74 略高于它（原视频里是 ~4.87），说明初始预测还带点"偏差"（在错误方向上自信）。这个差距由随机初始化决定，训练会把它纠正过来。

generate：采样续写

```

python
def generate(self, idx, max_new_tokens):
    for _ in range(max_new_tokens):
        logits, loss = self(idx) # 只看最后一个 token 就够 (bigram)

```

```
logits = logits[:, -1, :] # (B, C)
probs = F.softmax(logits, dim=-1) # (B, C)
idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)
idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
return idx
```

流程：取最后一个位置的 logits → softmax 变概率 → torch.multinomial 按概率采样 1 个 token → 拼回序列 → 重复 max_new_tokens 次。

- △ generate 里 self(idx) 不传 targets, 所以 forward 里 targets=None 分支要返回 loss = None ("有 targets 算 loss, 没 targets 只给 logits")。
- 为什么 bigram 也要把整个序列喂进去再只取最后一位？因为我们想让 generate 函数保持不变, 等以后模型真的会看历史了, 这个函数就"自动变聪明"。现在看起来很傻, 将来会派上用场。

训练循环 + AdamW

标准的三大步, 以及优化器换成 AdamW :

```
python
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for iter in range(max_iters):
    xb, yb = get_batch('train')
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

- 三大步顺序是固定的：zero_grad() (清空上一步梯度) → backward() (算梯度) → step() (更新参数)。
- AdamW vs SGD : makemore 前几课用的都是最朴素的 SGD (随机梯度下降) ; AdamW 自带自适应学习率, 对初始 lr 的敏感度比 SGD 低。实践中大模型常用 3e-4 量级作为起点, 小实验可以试更大或更小的值。

训练 1500 步的真实日志：

```
——— 训练 (AdamW, lr=0.01, 1500 步) ———
step 0: train loss 4.7618, val loss 4.7741
step 500: train loss 2.5877, val loss 2.6073
step 1000: train loss 2.4985, val loss 2.5239
step 1499: train loss 2.4972, val loss 2.5017
```

- 训练后 val loss ≈ 2.50 。这比初始的 ~ 4.8 好很多, 但离"好语言模型"还很远。(原视频跑出的也是 ≈ 2.5 , 不同超参/种子会有小差异。)

训练后的生成结果 (200 个字符) ——已经有零星的英文碎片, 但没有真正的词汇/语法结构：

```
CI n:
Wiwist Rorer boomatowig d:
Son cotheraris
STun:
S:
Th y pr;
```

Mims Fo;

Bony s,

Stece butis y DU\$muou s mularet w ke s ur, o aly agre d ndont h seld'lysen t's 'd ffere bl mureligescheple ord otak

> 一眼看出问题：bigram 只用了最后一个字符，上下文完全被浪费。比如它看到 Th 能猜到下一个是 e，但要猜出"这句莎士比亚在说什么"，必须看更长的历史。

这就是 Transformer 要解决的问题：让 token 之间互相交流，根据上下文做更好的预测。

学完这部分你能...

- 说清楚 ChatGPT = 语言模型 = 逐 token 预测，GPT 三个字母的含义
- 讲出 Transformer 的起源（2017 《Attention is All You Need》，机器翻译背景）
- 读懂并手写字符级 encode/decode，理解它与 BPE/sentencepiece 的取舍
- 划分 train/val (90/10)，理解验证集测过拟合的作用
- 讲清 block_size/batch_size、一个 chunk 含多样本、get_batch 采样
- 实现 Bigram 基线，理解交叉熵为什么 reshape 成 (B*T, C)
- 看懂 generate 采样流程，用 AdamW 训练到 loss ≈ 2.5

课后练习

Q1: 为什么训练时要让模型覆盖"上下文长度从 1 到 block_size"的所有情况？

A: 有两个原因。一是效率：一个 chunk 里的每个位置都能贡献一个训练样本，白白浪费很可惜。二是泛化：推理时我们可能从一个字符就开始生成，模型必须习惯各种长度的上下文。如果只训练"固定长度 block_size"的样本，推理开头那些短上下文它就应付不来。

Q2: 为什么交叉熵要把 logits 从 (B, T, C) reshape 成 (B*T, C)？targets 为什么也要 reshape？

A: PyTorch 的 `F.cross_entropy` 期望的输入是 (样本数, 类别数) 和 (样本数,)。我们把 (batch, time) 两个维度合并成一个"样本数"维度，每个位置的 65 维 logits 成为一个独立的分类样本，targets 也摊平成 (B*T,)。view 只改视图不拷贝内存，很高效。

Q3: 假如把 tokenizer 换成 BPE（词表 50K），其它训练代码要改吗？block_size 应该变吗？

A: encode/decode 换掉后，训练/模型代码几乎不用改，因为模型只依赖 vocab_size（由新词表决定，变为 ~50K）和 token 序列。但通常应该增大 block_size：BPE 的序列更短，相同的上下文长度能覆盖更多"真实内容"；同时因为词表变大、每个位置的信息量变大，也可能需要更大的模型。这就是"词表大小 vs 序列长度"的权衡在实践中的体现。

课后作业

完成本章后，去 Assignment 6 完成题 1（Tokenizer）和题 2（Dataloader）：

[Assignment 6](../../assignments/assignment_6/)

下一步

数据管线通了、Bigram 基线也建好了 (val loss ≈ 2.5)。下一步，我们进入本课的核心——attention。先用一个玩具演示"用矩阵乘法做加权聚合"这个数学技巧，再实现真正的 self-attention。

[02 — Attention 从零开始](02_attention_from_scratch.md)

02_attention_from_scratch

02 — Attention 从零开始：数学技巧、Self-Attention 与 6 条笔记

> 本课核心：attention 就是"token 之间按重要程度通信"。我们从最笨的 for 循环平均，一路进化到数据依赖的 self-attention。

前置知识

本章需要你已掌握：

- 01 章全部内容：字符级 Tokenizer、Dataloader、Bigram 基线、交叉熵 loss
- 矩阵乘法直觉：理解 `torch.matmul` 和"一行乘一列"的内积运算

从 Bigram 的局限出发

上一章 Bigram 训练到 $\text{val loss} \approx 2.50$ ，生成结果只有零星碎片。原因很直白：token 之间完全不交流，每个位置只用自己的身份预测下一个字符。

要让 token 交流，最简单的方式是——让每个 token 看看过去的信息，聚合起来帮助预测。但怎么聚合、聚合多少，很有讲究。

这一章先玩一个数学技巧（`[03_attention_trick.py](../scripts/03_attention_trick.py)`），再实现真正的 self-attention（`[04_self_attention.py](../scripts/04_self_attention.py)`）。

Part A：attention 的数学技巧

玩具问题设定

设一个张量 x ，形状 (B, T, C) ：B 个序列、T 个 token、每个 token 有 C 维信息。我们想让每个 token 聚合"自己及之前所有 token"的信息。

```
python
B, T, C = 4, 8, 2
x = torch.randn(B, T, C)
```

注意：token 只能看过去和自己，不能看未来——因为我们要预测未来，不能提前偷看答案。

v1：for 循环 bag-of-words 平均（最弱，但直观）

对每个序列、每个时间步，把当前及之前的 token 求平均：

```
python
xbow = torch.zeros((B, T, C))
for b in range(B):
```

```
for t in range(T):
xprev = x[b, :t + 1] # (t+1, C) : 当前及之前的 token
xbow[b, t] = xprev.mean(0) # 对时间维求平均 → bag of words
```

- bag of words (词袋) : 把一堆向量简单地平均成一条特征向量。这是最弱的聚合方式——把所有 token "糊"在一起, 丢掉了它们的位置和顺序信息。但它直观, 是理解一切的起点。

v2 : 用矩阵乘法做加权聚合 (数学技巧)

核心技巧: 加权聚合可以用矩阵乘法实现。如果有个矩阵 `wei`, 其中 `wei[t, i]` 表示"第 `t` 个 token 聚合第 `i` 个 token 的权重", 那么 `wei @ x` 就是在做加权求和。

用 `torch.tril` 造一个下三角全 1 矩阵:

```
python
tril = torch.tril(torch.ones(T, T))
```

运行结果 (T=8 的下三角矩阵, 1 表示聚合该 token, 0 表示忽略) :

```
tensor([[1., 0., 0., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0., 0., 0.],
        ...
        [1., 1., 1., 1., 1., 1., 1., 1.]])
```

- 下三角的"几何含义": 第 `t` 行只在 $\leq t$ 的位置是 1, 其它是 0——未来 (`t` 之后) 的 token 不参与聚合。这是自回归 (autoregressive) 的雏形。

把每行归一化成和为 1 (变成"平均"), 再乘上 `x` :

```
python
wei = tril
wei = wei / wei.sum(1, keepdim=True) # 每行归一化成和为 1 → 变成"平均"
xbow2 = wei @ x # (T,T) @ (B,T,C) → 批矩阵乘法 (B,T,C)
```

归一化后的权重 (每行和为 1) :

```
tensor([[1.0000, 0.0000, ...],
        [0.5000, 0.5000, ...],
        [0.3333, 0.3333, 0.3333, ...],
        ...
        [0.1250, 0.1250, 0.1250, 0.1250, 0.1250, 0.1250, 0.1250, 0.1250]])
```

- $(T,T) @ (B,T,C)$: PyTorch 看到维数不匹配, 会把它当批矩阵乘法——在 batch 维上对每个序列独立做 $(T,T) @ (T,C)$, 得到 (B,T,C) 。这正是"batch 间不通信"的数学基础。

- 第 `t` 行是 $1/t$ 均匀分布 → 第 `t` 个 token 的结果等于前 `t` 个 token 的平均, 和 `v1` 完全一样。用 `torch.allclose` 验证:

`v1` 与 `v2` 等价 (`torch.allclose`): True

v3 : softmax 版本 (亲和力 + 遮罩, 重点记忆)

`v3` 换一个视角: 把权重矩阵当成"亲和力 (affinity)", 初始全 0 (无差异), 用 `masked_fill` 把未来置为 `-inf`, 再让 softmax 把它归一化成"概率":

attention 是对"一组向量"做操作，默认不知道每个 token 在序列里的位置。所以我们用第二张 embedding 表给每个位置也学一个向量：

```
python
self.position_embedding_table = nn.Embedding(block_size, n_embd)
```

forward 里：

```
tok_emb = self.token_embedding_table(idx) # (B,T,C)
pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
x = tok_emb + pos_emb # 广播相加 (B,T,C)
```

- 广播 (broadcasting)：(B,T,C) + (T,C)，PyTorch 把 (T,C) 右对齐，前面补一个维度变成 (1,T,C)，再沿 batch 维广播 → 每个 token 的向量 = "我是谁" (token embedding) + "我在哪" (position embedding)。
- 为什么 Bigram 阶段这个位置信息没用？因为 bigram 是平移不变的 (translation invariant)，token 在第 5 位还是第 2 位无所谓。等 attention 真正"看上下文"了，位置信息就开始起作用——因为"第 3 个位置出现的是元音"和"第 7 个位置出现的是元音"，在聚合时意义不同。

Head：单个 self-attention 头

现在实现本课核心。每个 token 发出三个向量：

- query (q)：我在找什么？
- key (k)：我有什么？
- value (v)：如果你觉得我有趣，我把什么传达给你？

亲和力 = query 与所有 key 的内积，然后遮罩、softmax、按权重聚合 value：

```
python
class Head(nn.Module):
    """一个自注意力头：每个 token 发出 query (找什么)、key (有什么)、
    value (若有趣就传达什么)，按亲和力聚合过去的信息"""
    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
```

tril 不是可训练参数，用 register_buffer 注册（会随模型移动设备）

```
self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

def forward(self, x):
    B, T, C = x.shape
    k = self.key(x) # (B,T,head_size)
    q = self.query(x) # (B,T,head_size)
```

亲和力 = query 与所有 key 的内积 → 数据依赖

$wei = q @ k.transpose(-2, -1) * k.shape[-1]^{-0.5}$ # (B,T,T), scaled

遮罩：未来不能看向过去（decoder 三角遮罩）

```
wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
wei = F.softmax(wei, dim=-1) # 每行归一化成概率 (B,T,T)
v = self.value(x) # (B,T,head_size)
out = wei @ v # 加权聚合 → (B,T,head_size)
return out
```

拆解关键几行：

- $q @ k.transpose(-2, -1) : (B, T, head_size) @ (B, head_size, T) \rightarrow (B, T, T)$ 。每个 token 的 query 和所有 token 的 key 做内积，得到一个 $T \times T$ 的亲合力矩阵。
- $* k.shape[-1]^{-0.5}$ ：即除以 $\sqrt{head_size}$ ，scaled attention（笔记⑥详述）。
- `masked_fill(self.tril[:T, :T] == 0, float('-inf'))`：把未来置 -inf，softmax 后它们变成 0。
- `wei @ v`：按亲合力对 value 加权求和。
- `bias=False`：K/Q/V 只是投影，通常不加偏置。
- `register_buffer('tril', ...)`：tril 不是可训练参数，但它必须作为模块的一部分（这样 `to(device)` 时它会跟着走）。这是 PyTorch 的 buffer 机制。

> 你可以把 x 想成 token 的"私密信息"；为了这个头的通信，token 额外发布"介绍信"：key（我有什么）、query（我找什么）、value（若你对我感兴趣，你会从我这儿拿到的东西）。聚合发生时，聚合的是 value，不是原始的 x 。

数据依赖的亲合力：实跑演示

[04_self_attention.py](../scripts/04_self_attention.py) 里用随机输入实跑了一个 Head(`n_embd`)，打印出亲合力矩阵（每行 = 该 token 对过去各 token 的注意力权重）：

亲合力矩阵（每行 = 该 token 对过去各 token 的注意力权重）：
 tensor([[1.0000, 0.0000, ...],
 [0.5141, 0.4859, ...],
 [0.3271, 0.3374, 0.3356, ...],
 ...
 [0.1136, 0.0961, 0.1207, 0.0845, 0.1033, 0.1394, 0.1377, 0.2048]])

注意这和 Part A 的均匀分布 ($1/t$) 完全不同：权重是数据依赖的，有的 token 被重视（如 0.38、0.31），有的被冷落（如 0.06）。这就是 attention 和"简单平均"的本质区别。

scaled attention：为什么除以 $\sqrt{head_size}$

```
python
vals = torch.tensor([0.1, -0.2, 0.3, -0.1, 0.2])
F.softmax(vals, dim=-1).tolist()
```

→ [0.2047, 0.1516, 0.2500, 0.1676, 0.2262] 接近 0 的小值
softmax（扩散）

```
F.softmax(vals * 8, dim=-1).tolist()
```

→ [0.1180, 0.0107, 0.5847, 0.0238, 0.2627] 放大 8 倍后
softmax（尖锐）

- 同样的值放大 8 倍，softmax 就从一个"相对均匀"的分布，变成"几乎只认最大值"的 one-hot 分布。

- 如果输入是 unit gaussian（零均值、单位方差），那么 q 和 k 的内积（wei）方差大约是 head_size。head_size 越大，wei 越尖锐。除以 $\sqrt{\text{head_size}}$ 让方差回到 ≈ 1 。
- $\triangle$ 初始化时我们不想让每个 token 只聚合一个 token（softmax 太尖锐）——我们希望一开始是“广撒网”的扩散分布，让网络自己学会该聚焦谁。所以缩放是必需的。

插入单头 self-attention 到网络

把 Head 插进语言模型（[04_self_attention.py](../scripts/04_self_attention.py)，这里类名沿用 BigramLanguageModel）：

```
python
class BigramLanguageModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.sa_head = Head(n_embd) # 单头, head_size = n_embd
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok_emb = self.token_embedding_table(idx)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))
        x = tok_emb + pos_emb
        x = self.sa_head(x) # 自注意力
        logits = self.lm_head(x) # (B,T,vocab_size)
        ...
```

- $\triangle$ generate 里必须裁剪 idx：位置表只有 block_size 个位置，生成会不断把新 token 拼进序列，一旦超过 block_size 就会越界。

```
python
def generate(self, idx, max_new_tokens):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -block_size:] # 只保留最后 block_size 个
    ...
```

训练 3000 步的真实日志（val loss 从 ~ 2.5 降到 ≈ 2.39 ）：

—— 训练 (AdamW, lr=0.003, 3000 步) ——

```
step 0: train loss 4.2351, val loss 4.2337
step 500: train loss 2.5160, val loss 2.4975
step 1000: train loss 2.4407, val loss 2.4234
step 1500: train loss 2.4328, val loss 2.4308
step 2000: train loss 2.3907, val loss 2.4244
step 2500: train loss 2.3784, val loss 2.4247
step 2999: train loss 2.3812, val loss 2.3871
```

生成结果（300 个字符）——文本开始有“词组”的影子了：

```
OMONofr atre kchen, ty yed wine nd hiche arsthitha heap's sl lis min ius m:
Wh SOPULorim; fome bet an sur t thire bes, wO!
TORDO CESwi, ous, achirery win-
```

Torsw ps, anst bitheed I apar:

...

- 单头 self-attention 让 token 开始按"数据依赖的亲合力"通信，val loss 从 bigram 的 ≈ 2.5 降到 ≈ 2.39 。还差得远，但方向对了。

6 条 attention 笔记（本课核心，逐一展开）

Karpathy 在视频里用 6 条笔记总结了 attention 的本质。这里全部展开：

笔记 1：attention 是通信机制

attention 就是一个有向图上的通信：节点（token）通过有向边，按权重聚合指向自己的节点信息。

有向图：边从"被看"的节点指向"观看"的节点
（每个节点聚合所有"指向自己"的节点的信息）

① → ② → ③ → ④ → ⑤ → ⑥ → ⑦ → ⑧

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

自己 ①② ①②③ ①②③④ ... ①..⑦ ①..⑧

只看 只看 只看 （自回归：第 t 个节点

自己 前2个 前3个 只聚合自己和之前的节点）

在我们语言建模的例子中，图是"自回归"的：第 t 个节点只看自己和之前的节点。但原则上 attention 可以作用在任意有向图上——它只是一个通用的通信机制。

笔记 2：attention 无空间概念，作用在"集合"上

attention 默认把输入当一组向量（set）处理，节点之间没有位置感。这正是我们前面加"位置编码"的原因——把"我在哪"的信息显式加到向量里。

- 对比 卷积（Part

5）：卷积有非常具体的空间性——卷积核按空间位置滑动，它天然知道邻域布局。attention 没有这个先天属性，想要"空间"，就得自己加上。

- 这也说明 attention 不关心 token 的绝对位置，只关心它们之间的信息关系——这是它强大（能适配任意序列）的原因，也是它"需要位置编码"的原因。

笔记 3：batch 之间不通信

批矩阵乘法 $\text{wei} @ \text{v}$ 在 batch 维上各自独立执行。batch 里的 32 个序列是 32 个互不相关的"图"，每个图内部 8 个节点互相通信，图与图之间绝不交流。

- 我们可以把它看作"4 个独立的池子，每个池子里 8 个节点"（batch_size=4 时）。它们共享同一套权重，但数据完全隔离。

笔记 4：decoder block 用三角遮罩；encoder block 全连通

我们实现的是 decoder block：用 `masked_fill(self.tril == 0, -inf)`

保证未来不看向过去。这叫自回归——预测下一个字符时不能提前看到答案。

但这不是唯一选择。如果做情感分析之类任务，让所有 token 互相看完全没问题（甚至更好）：

decoder block（我们）：encoder block：

未来不看向过去（三角遮罩） 所有节点全连通（删除遮罩行）

> 实现 encoder 只需要删除遮罩那一行。attention 本身不关心连边方式，它支持任意拓扑。我们讲完 6 条笔记后会回来展开 encoder/decoder 的完整图景（见第 04 章）。

笔记 5：attention vs self-attention vs cross-attention

- attention：最一般的机制——"按亲和力聚合"。
- self-attention：K、Q、V 全部来自同一个 X（"自己看自己"）。我们实现的就是它。
- cross-attention：Q 来自 X，但 K、V 来自另一个独立的外部源（比如 encoder 的输出）。用于"从旁边拉信息进来"（如翻译时读法语、写英语）。

self-attention: cross-attention:

X —► K encoder 输出 —► K / V

X —► Q → wei@v X (decoder) —► Q → wei@v

X —► V

- 我们这种"只有自己看自己"的注意力，叫 self-attention；但 attention 本身远比这通用。

笔记 6：scaled attention——除以 $\sqrt{\text{head_size}}$ 控制方差

前面已用实际输出演示过：输入 unit gaussian 时， $q@k$ 的方差 $\approx \text{head_size}$ ，值会随 head_size 变大而尖锐；softmax 会把尖锐的值推向 one-hot，导致"每个 token 只聚合一个 token"。除以 $\sqrt{\text{head_size}}$ 把方差拉回 ≈ 1 ，让初始化时的注意力保持扩散（每个 token 雨露均沾）。

$\text{wei} = q @ k.\text{transpose}(-2, -1) * k.\text{shape}[-1]^{-0.5} \# k.\text{shape}[-1] = \text{head_size}$

- 一句话：缩放 = 在初始化时保护 softmax 的"温和"。训练中网络自己学会该专注谁，但起点必须温和。

Multi-Head：多头并行

分组卷积的类比

attention 很好，但一个头只有一个通信通道。token 想找元音、想找位置、想找标点——这么多"话题"挤在一个 32 维通道里太挤。解决办法：多个头并行，每个头一个小通道。

类比：卷积里的 group convolution（分组卷积）——不做一个大的卷积，而是分成多组小卷积。多头 self-attention 也是同样的思想。

python

```
class MultiHeadAttention(nn.Module):
```

```
def __init__(self, num_heads, head_size):
```

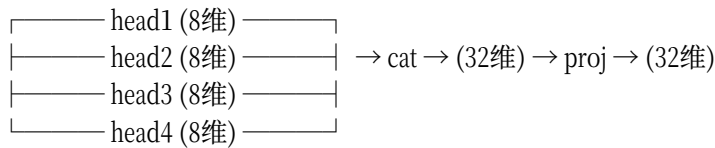
```
    super().__init__()
```

```
    self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
```

```
    self.proj = nn.Linear(head_size * num_heads, n_embd) # 投影回残差通路
```

- `head_size = n_embd // n_head`：把 `n_embd` 均分给 `n_head` 个头。例如 `n_embd=32, n_head=4` → 每个头 `head_size=8`。每个头输出 8 维，4 个头拼接回 32 维。
- `proj`：把拼接后的 (`head_size * num_heads`) 维投影回 `n_embd` 维，为后面接残差连接做准备（第 03 章会用到）。注意：现在还没有残差连接，`proj` 暂时只是一个形状变换；等 03 章加入残差连接后，它才真正发挥“把输出投影回残差通路”的作用。

多头: 4 个 8 维并行通道, 拼接回 32 维



[05_multihead_feedforward.py](../scripts/05_multihead_feedforward.py) 的 Phase 1 用 `nn.Sequential(MultiHeadAttention(n_head, head_size))` 训练 400 步：

• △ 我们的CPU小规模(400步)下多头 val loss ≈ 2.45 , 和单头脚本 04 的 ≈ 2.39 在同一量级——多头"多个通信通道"的价值, 在数据量更大、网络更深时才充分显现(原视频中 $2.4 \rightarrow 2.28$)。所以不要只看这一个数字, 看它带来的结构性收益: 更多独立的通信通道, 能同时捕捉多种"话题"。

- 用三种数学等价的方式实现"过去 token 的加权聚合" (for 循环 / 矩阵乘法 / softmax)
- 解释为什么下三角遮罩保证"未来不看向过去"
- 写出单头 self-attention (query/key/value、亲和力、缩放、遮罩、register_buffer)
- 解释为什么 attention 需要位置编码, 以及 (B,T,C)+(T,C) 的广播机制
- 把 6 条 attention 笔记逐条讲清楚
- 实现 Multi-Head (分组卷积类比、head_size = n_embd // n_head、proj)

A: 如果输入是 unit gaussian, $q @ k$ 的方差约为 `head_size`。`head_size` 越大, `wei` 里正负值越大, softmax 会把每一行推向 one-hot——每个 token 几乎只聚合一个

token。初始化时我们想要的是"扩散"的注意力，这样才能让梯度均匀流动、网络自主学习聚焦谁。缩放让 wei 的方差回到 ≈ 1 ，保护 softmax 的温和。

Q3: cross-attention 和 self-attention 的区别是什么？为什么机器翻译（法语→英语）需要 cross-attention？

A: self-attention 的 K/Q/V 全部来自同一个 X（自己看自己）；cross-attention 的 Q 来自当前序列（decoder），而 K/V 来自另一个外部源（encoder 编码后的法语）。翻译时，decoder 要生成英语，但必须"参考"已经读完整句的法语信息——这个"从旁边拉信息"的动作就是 cross-attention。我们第 04 章会详细展开。

课后作业

完成本章后，去 Assignment 6 完成题 3（Bigram 模型）和题 4（单头 Self-Attention）：

[Assignment 6](../../assignments/assignment_6/)

下一步

single-head 和 multi-head 都实现了，但网络还很浅（token 看完彼此后立刻就去预测）。下一步我们给 token 一个"思考"的步骤（FeedForward），再用残差连接和 LayerNorm 让网络能堆到很深。

[03 — Transformer Block](03_transformer_block.md)

03_transformer_block

03 — Transformer

Block：FeedForward、残差连接、LayerNorm、Scale Up

> 把"通信"和"计算"配对成 Block，用残差连接 + LayerNorm 让它能叠得很深，最后 scale up 成一个真正的 mini-GPT。

前置知识

本章需要你已掌握：

- 02 章全部内容：attention 数学技巧、self-attention 单头、multi-head、6 条 attention 笔记
- Part 4 加法节点的梯度规则："加法把梯度均分给两个分支"——讲残差连接时直接用到
- Part 3 BatchNorm：训练/推理两态、归一化"列"——讲 LayerNorm 时会和它对比

从"看完就走"到"看完再想想"

上一章，token 通过（multi-head）self-attention 完成通信——互相看了看对方。但紧接着就去做预测，token 没时间消化看到的東西。

论文里的 Transformer Block 其实是"两段式"的：先通信，再计算，然后整个 Block 重复很多次。

```
| |
| ① 通信 (communication) | ← Multi-Head Self-Attention
| ② 计算 (computation)   | ← FeedForward (MLP)
```

(Block 重复 `n_layer` 次, 叠成深层网络)

FeedForward：通信之后，各自思考

attention 让 token 互相交换信息；FeedForward 则让每个 token 独立地"思考"——它是对每个 token 分别做的 MLP，token 之间不交流。

```
python
class FeedForward(nn.Module):
    """逐 token 的前馈网络：通信之后"各自思考"
    内层 4×n_embd (论文 512→2048 的 4 倍规律)"""
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
        )
    def forward(self, x):
        return self.net(x)
```

- 结构：Linear($n_embd \rightarrow 4n_embd$) $\rightarrow$ ReLU $\rightarrow$ Linear($4n_embd \rightarrow n_embd$)。内层扩到 4 倍，这是论文里的规律：512 $\rightarrow$ 2048。你可以把它理解为"先是开大的空间里思考，再压缩回原尺寸"。
- $\triangle$ 它作用在 (B, T, C) 的最后一个维度上——每个 token (每个 (b, t) 位置) 独立经过同一个 MLP。这就是"per-token" (逐 token) 的含义。

我们的脚本 [05_multihead_feedforward.py](../scripts/05_multihead_feedforward.py) 里 Phase 2 在多头后面接上 FeedForward 再训练：

```
[Phase1 多头] train loss 2.3813, val loss 2.4545
[Phase2 +前馈] train loss 2.4372, val loss 2.5006
```

- $\triangle$ 注意 Phase 2 的 val loss (≈ 2.50) 比 Phase 1 还略高——这是少步数 + 前馈层更"深"导致收敛变慢，多跑步数就会降下来 (原视频中 2.28 $\rightarrow$ 2.24)。不要被单个数字骗了，看趋势、看结构收益。

残差连接：梯度超高速公路

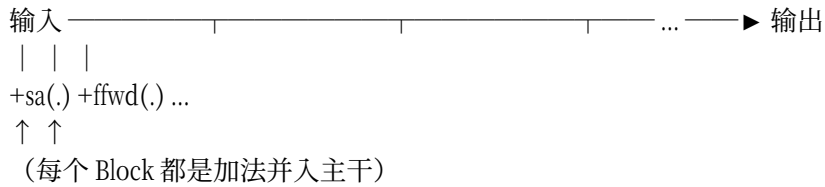
现在把 Block 叠起来——问题来了：网络变深之后，优化变得困难。论文里有两个"特效药"，残差连接是第一个。

核心思想

```
python
```

```
x = x + self.sa(x) # 通信后加回残差通路
x = x + self.ffwd(x) # 思考后加回残差通路
```

可以这样可视化：从输入到输出，数据只通过加法走一条"主干道"，每个 Block 是主干道旁边"岔出去"的小作坊——做完事再加回来。



为什么加法和梯度有关？

回想起 Part 4（手动反向传播）：加法节点会把梯度均分给它的两个输入分支。这意味着：

- 反传时，残差通路 ($x + f(x)$ 中的 x) 的梯度是 1（恒等映射），所以梯度可以沿残差通路从 loss 直达输入、不衰减。加上子模块那一支的梯度，总梯度 = 1 + 子模块梯度。初始化时子模块很小，梯度近似为 1。
- 随着训练，残差块逐步"上线"、逐步贡献，但梯度高速公路始终畅通。

> 这就是"梯度超高速公路（gradient super highway）"。它让深层的 Transformer 可优化。没有它，几层 Block 叠起来就很难训练了。

我们的脚本 [05_multihead_feedforward.py](../scripts/05_multihead_feedforward.py) 的 Phase 3（BlockNoLN）实跑结果——残差连接带来最明显的一跃：

[Phase3 + 残差] train loss 2.1547, val loss 2.2324

—— 演进对比 ——

Script 4 单头 self-attn: ~2.4

Phase 1 多头并行: 2.4545

Phase 2 + 前馈网络: 2.5006

Phase 3 + 残差连接: 2.2324

- Phase 3 用 $x = x + \text{self.sa}(x)$; $x = x + \text{self.ffwd}(x)$ 把多头 + 前馈 + 残差全部组合，val loss 一下从 ~2.5 降到 ≈ 2.23。这是本脚本最明显的一跃，正是"深层网络能优化了"的效果。

proj：投影回残差通路

MultiHeadAttention 末尾的 `proj = nn.Linear(head_size * num_heads, n_embd)` 就是把拼接后的多头输出投影回 `n_embd` 维，好让 $x + \text{sa}(x)$ 形状匹配。FeedForward 末层 `Linear(4n_embd → n_embd)` 同理。

LayerNorm：让更深的网络稳定（与 BatchNorm 对比）

第二个"特效药"是 LayerNorm（层归一化）。它和 Part 3 的 BatchNorm 关系密切，但有几个关键差异。

BatchNorm 归一化"列"，LayerNorm 归一化"行"

输入 (B, C) = (4, 5) :

BatchNorm : 对每一"列" (每个特征通道, 跨 batch 的样本) 归一化

↓ ↓ ↓ ↓ ↓ (归一化的是竖着的列)

LayerNorm : 对每一"行" (每个样本的特征向量) 归一化

→ → → → → (归一化的是横着的行)

LayerNorm 在我们的 (B, T, C) 上, 就是对每个 token 的 n_embd 维特征归一化 (把 batch 和 time 都当作"样本"维)。

[06_layernorm_transformer.py](../scripts/06_layernorm_transformer.py) 里用 $x = \text{torch.randn}(4, 5)$ 实践演示 :

———— LayerNorm vs BatchNorm ————

原始 x 每行 mean: tensor([-0.6402, 0.2848, -0.3360, 0.3817])

原始 x 每行 std: tensor([0.8810, 0.9499, 1.3110, 1.7137])

LayerNorm 后每行 mean: tensor([...e-08...]) (≈ 0)

LayerNorm 后每行 std: tensor([1.1180, 1.1180, 1.1180, 1.1180]) (≈ 1)

- 每行归一化后 $\text{mean} \approx 0$ 、 $\text{std} \approx 1$ 。而且所有行共享同一个 std (1.1180) , 说明 LayerNorm 是逐行独立归一化的。
- ⚠ 注意 std 是 1.1180 而不是精确的 1.0——这是因为 PyTorch 的 `nn.LayerNorm` 默认用无偏方差 (除以 $n-1$ 而非 n) , 5 维时 $\text{sqrt}(5/4) \approx 1.118$ 。这是数值细节, 不影响理解。

与 BatchNorm 的三个关键差异

维度	BatchNorm (Part 3)	LayerNorm (本课)
归一化方向	跨 batch 的"列" (特征通道)	per-token 的"行" (n_embd 特征)
running buffer	需要 EMA 的 <code>running_mean/var</code>	不需要, 无训练/推理区分
可学习参数	γ (gamma) / β (beta)	同样保留 γ (gamma) / β (beta)

- LayerNorm 没有 running buffer, 因为它归一化时不依赖其它样本——训练和推理行为完全一致, 代码更简单 (不需要 `model.train()/model.eval()` 切换)。这是它在 Transformer 里比 BatchNorm 更合适的重要原因。
- 归一化后同样保留可学习的 γ 和 β , 让网络能"撤掉"归一化效果、学自己想要的分布。

Pre-norm 结构

论文原版是 post-norm (先 attention, 后 LayerNorm)。现在更常见的做法是 pre-norm : 先 LayerNorm, 再进入 attention/ffwd。本课采用 pre-norm (也几乎成了现代 Transformer 的标准)。

```
python
class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
```

```
x = x + self.sa(self.ln1(x)) # pre-norm : 先 LN 再 attention
x = x + self.ffwd(self.ln2(x)) # pre-norm : 先 LN 再 fwd
return x
```

• ⚠ 顺序很重要： $x = x + \text{self.sa}(\text{self.ln1}(x))$ 是"先归一化 x ，再做 attention，再加回来"。别写成 $\text{self.sa}(x)$ 再归一化——那是 post-norm。

完整 decoder-only Transformer

[06_layernorm_transformer.py](../scripts/06_layernorm_transformer.py) 把 Block 叠两层，并加上最终 LayerNorm ln_f （在 lm_head 之前，把最终特征归一化后再投影到词表）：

```
python
class BigramLanguageModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # 最终 LayerNorm (lm_head 之前)
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok_emb = self.token_embedding_table(idx)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))
        x = tok_emb + pos_emb
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)
        ...
```

完整的 decoder-only Transformer 管线：

```
token 编码 + 位置编码
→ N × Block(残差 + 多头注意力 + 前馈 + LayerNorm(pre-norm))
→ ln_f
→ lm_head → logits
```

2 层 Block + LayerNorm 训练 1200 步的真实日志：

```
——— 训练 (n_layer=2, AdamW, lr=0.003, 1200 步) ———
step 0: train loss 4.3437, val loss 4.2889
step 400: train loss 2.4303, val loss 2.4364
step 800: train loss 2.3276, val loss 2.3338
step 1199: train loss 2.2512, val loss 2.2340
```

• 最终 val loss ≈ 2.23 （和脚本 05 Phase 3 的 2.23 巧合地接近）。这里 $n_{\text{layer}}=2$ 已经比脚本 05 的单层更深，说明 LayerNorm 的主要价值不是"降 loss"，而是让更深的网络也能稳定优化。

• 参考原视频的演进（完整超参）：bigram 2.5 $\rightarrow$ 单头 2.4 $\rightarrow$ 多头 2.28 $\rightarrow$ 前馈 2.24 $\rightarrow$ 残差 2.08 $\rightarrow$ +LayerNorm 2.06。我们的 CPU 小规模数字（2.50 $\rightarrow$ 2.39 $\rightarrow$ 2.45/2.50/2.23 $\rightarrow$ 2.23）趋势一致、绝对值有差异（超参/种子不同），看趋势别背数字。

生成结果（400 个字符）——已经有点像"对话体"了：

Tit ILA:

Uy?

BF

PUY:

Foured

She manst I sipre.

GCAPEErq:

Thondl with youns?

CILA:

Am fore therars.

The dall ill sit hem will elpimpke a'd me chat fraser 'at; he lich he blavk bed;

...

Dropout：随机关掉一些神经元

要 scale up 之前，先加一个正则化技巧——Dropout（[Srivastava et al., 2014](#)）。它在前向/反传时随机把一部分神经元（或注意力权重）置零：

- 每次前向/反传的"置零掩码"都不同 → 等价于训练了一堆子网络的集成。
- 测试时所有神经元全开 → 相当于把那一堆子网络合并成一个集成。
- 一句话：正则化，防止过拟合。

python

Head 里：softmax 后随机屏蔽部分注意力

```
wei = F.softmax(wei, dim=-1)
```

```
wei = self.dropout(wei) # 随机阻止一些节点通信
```

MultiHeadAttention：残差连接前

```
out = self.dropout(self.proj(out))
```

FeedForward：残差连接前

```
nn.Linear(4 * n_embd, n_embd),
```

```
nn.Dropout(dropout),
```

- $\triangle$ 放置位置：残差连接之前（attention 输出、feedforward 输出），以及 softmax 之后的注意力权重。这些是 Dropout 在 Transformer 里的典型位置。

Scale Up：超参数放大 + 参数统计 + 生成

现在把前面所有组件拼成完整的

GPTLanguageModel ([07_scaleup_generate.py](../scripts/07_scaleup_generate.py)，与 gpt.py 收敛一致)，加 Dropout 和更好的初始化，然后放大超参。

参数统计

```
python
model = GPTLanguageModel().to(device)
n_params = sum(p.numel() for p in model.parameters())
```

实跑输出：

——— 模型 ———

缩小版超参: batch=16, block=64, n_embd=64, n_head=4, n_layer=2, dropout=0.2
参数量: 112,193 = 0.112 M

- `sum(p.numel())` 把所有参数张量的元素数加起来，/1e6 就是百万为单位。我们的 CPU 缩小型只有 $\approx 0.11\text{M}$ 参数；原视频 GPU 完整版约 10M 参数。

超参数对比

超参	GPU 完整版（原视频，A100 ~15 分钟）	CPU 缩小型（我们，<30s）
batch_size	64	16
block_size	256	64
n_embd	384	64
n_head	6	4
n_layer	6	2
dropout	0.2	0.2
lr	3e-4	3e-4
max_iters	5000	150
val loss	1.48	≈ 2.80

- ⚠ 我们没有 GPU，所以主动缩小规模（更小的 n_layer/n_embd/block_size/步数），让它在 CPU 上 <30s 跑完。原视频里"2.07 $\rightarrow$ 1.48"那一步是靠 A100 + 完整超参烧出来的，CPU 上跑不了。

缩小型训练 150 步的真实日志：

——— 训练 (AdamW, lr=0.0003, 150 步) ———

step 0: train loss 4.1655, val loss 4.1661
step 100: train loss 2.9802, val loss 3.0016
step 149: train loss 2.7744, val loss 2.8044

- 150 步太少，val loss 只到 ≈ 2.80 ，但趋势对：如果给它更多步数 + 更大规模，就能逼近视频里的 1.48。脚本里已经把"完整超参 + 10M 参数 + 1.48"的说明打印出来了。

生成：从换行符开始

起始上下文是 `torch.zeros((1, 1))`，也就是一个 token：索引 0 = 换行符。这是个合理的"开场"。

```
python
context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(model.generate(context, max_new_tokens=500)[0].tolist()))
```

- 生成的 token 数可以随意加大：脚本里注释演示了 `max_new_tokens=10000` 并写进文件（`open('more.txt', 'w')...`），就能生成 1 万个字符的"伪莎士比亚"。

缩小型生成的 500 个字符（训 150 步，所以还比较乱——这是步数不足的表现，不是架构的错）：

```
'pi!QAz-nh ntt an ,eerooQLlhAFE,;rde ce toaise
Whe ninssete d ?JW.RCWeo,jmolshonulodioqosl mroQ:
TT osaOunoo:TcWVcevisn, e,raOoluncuz hndrAssmo at mi;
...
```

> 对比：原视频用完整超参生成 10K 字符，看起来就"很像莎士比亚了"（虽然读起来仍然无意义）——这是 scale 的力量。

学完本部分你能...

- 理解"通信（attention）vs 计算（FeedForward）"的分工，写出 per-token MLP
- 解释残差连接 = "梯度超高速公路"：加法均分梯度、梯度直达输入
- 对照 Part 3 讲清 LayerNorm vs BatchNorm 的差异（行列/无 running buffer/保留 γ β ）
- 写出 pre-norm 的 Block，理解它和论文 post-norm 的区别
- 组装完整的 decoder-only Transformer，统计参数量
- 解释 Dropout 为什么能正则化，以及它放在哪些位置
- 理解 CPU 缩小型与 GPU 完整版（1.48）的区别

课后练习

Q1: 为什么"残差连接 + 加法把梯度均分给两个分支"能帮助深层网络训练？

A: 反传时，加法节点的梯度会均分给它的两个输入。残差通路（ $x + \dots$ ）从 loss 到输入全程只有加法，梯度可以"跳过"所有残差块、几乎不衰减地直达输入——这就是梯度超高速公路。残差块刚初始化时贡献很小（近似等于没加），所以早期梯度畅通无阻；训练中残差块逐步"上线"。这样网络想学多深都不会"梯度传不到"。

Q2: LayerNorm 和 BatchNorm 的本质区别是什么？为什么 Transformer 里选 LayerNorm？

A: BatchNorm 对"列"归一化（跨 batch 的特征通道），需要维护 running buffer，训练/推理行为不同；LayerNorm 对"行"归一化（每个 token 的 `n_embd` 特征），不需要 running buffer，训练/推理无区别。Transformer 的序列长度（`T`）可变、且依赖样本内归一化，LayerNorm 因为不依赖其它样本、没有训练/推理两态，所以更合适、更简单。两者都保留可学习的 γ / β 。

Q3: 我们从 2.5（bigram）一路降到 2.23（2 层 Block），为什么 scale up 到 10M 参数能再降到 1.48？

A: scale up 同时增加了三样东西：模型容量（`n_embd` 32→384、6 层 Block、约 10M 参数）、上下文长度（`block_size` 8→256，模型能看到更长的"莎士比亚台词"）、训练步数与 batch（5000 步 × batch 64）。更大容量能记住更多规律，更长上下文能更好地预测下一个字符。Dropout 0.2 在放大后抑制过拟合。三者叠加，val loss 从 2.07 一路压到 1.48。当然这需要 A100 约 15 分钟——计算是 scale 的燃料。

课后作业

完成本章后，去 Assignment 6 完成题 5（完整 Transformer Block）：

[Assignment 6](../assignments/assignment_6/)

下一步

我们的 decoder-only Transformer

已经完整、能训练、能生成。最后一部分，我们把镜头拉远：我们实现的到底是整个 Transformer 的哪一半？另一半（encoder + cross-attention）长什么样？工业界的 nanoGPT 和 ChatGPT/GPT-3 是怎么从我们这 200 行代码走向生产的？

[04 — 超越 Transformer](04_beyond_transformer.md)

04_beyond_transformer

04 — 超越 Transformer：Encoder/Decoder、nanoGPT、回到 ChatGPT

> 我们把镜头拉远：我们的 mini-GPT 是完整 Transformer 的哪一半？工业界的 nanoGPT 怎么写？ChatGPT/GPT-3 是怎么从这 200 行代码走向生产的？

前置知识

本章需要你已掌握：

- 01-03 章全部内容：Tokenizer、Dataloader、Bigram、Self-Attention、Multi-Head、FeedForward、残差、LayerNorm、完整 decoder-only Transformer

> 本章是"全景回顾 + 展望"，不再有新代码实现，重在建立全局理解。

从"我们做了什么"出发

前 3 章我们用约 200 行代码训练了一个 decoder-only Transformer，能在 tiny Shakespeare 上生成"伪莎士比亚"。现在的问题：这和 2017 年那篇论文《Attention is All You Need》里的"Transformer"是同一个东西吗？

答案是：是，但不完整。论文画的是一个 encoder-decoder（编码器-解码器）架构，我们只实现了它的解码器（decoder）那一半。下面把这幅"官方全家福"补齐。

> 前置知识：本章需要你已掌握 01-03 章全部内容。此外，理解 encoder 特性时会用到 Part 5 的"卷积有空间性"概念（对比 attention 无空间概念），理解 LayerNorm 时会用到 Part 3 的 BatchNorm 概念（归一化方向对比）。

Encoder vs Decoder vs 完整架构

我们实现的：decoder-only

- 只有 decoder：带三角遮罩的自回归 Transformer。
- 没有 encoder、没有 cross-attention。

• 为什么？因为我们的任务是"无条件生成文本"——没有额外的"输入"需要去编码，只需要照着数据集"喋喋不休"。

论文里的：encoder-decoder（机器翻译）

原论文做的是机器翻译（法语 → 英语），所以需要两套 Transformer：

法语句子 "Je suis très heureux"

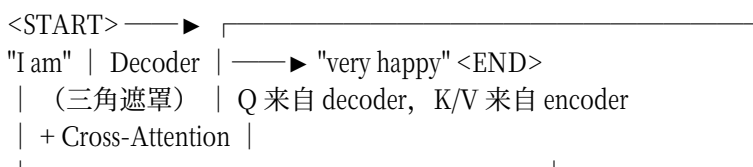
|



|

(编码后的法语表示)

| K/V



几个关键点：

- decoder 的特征 = 三角遮罩（自回归）：预测下一个词时不能看未来的答案。这就是"decoder"的定义，我们实现的就是它。
- encoder 的特征 = 删除遮罩行，所有节点互相通信：它要"通读"整句法语，不用自回归，所以可以让所有 token 互相看个够。
- cross-attention：decoder 生成时，Q 仍然来自 decoder 自己（"我想生成什么"），但 K/V 来自 encoder 的输出（"我参考的是已编码的法语"）。这就是把"读法语"和"写英语"接起来的桥梁。
- 特殊 token：<START> 放在生成序列开头（告诉模型"开始翻译了"），<END> 表示生成结束。这些是为任务新增的专用 token，不出现在自然文本中，但会被加入 tokenizer 的词表（有自己专属的 ID）。

decoder block（我们）：encoder block：

未来不看向过去（三角遮罩） 所有节点全连通（删除遮罩）

自回归、可采样 通读整句、提取表示

完整架构 = encoder + decoder + cross-attention（翻译等"条件生成"任务）

- 为什么 GPT 只用了 decoder？因为 GPT 是"文档补全器"——没有额外条件输入，只需要自回归生成。decoder-only 恰好就是生成语言模型最需要的形态。

nanoGPT 走读

Karpathy 把"生产级但极简"的代码放在了 [nanoGPT](https://github.com/karpathy/nanoGPT)：两个文件，各约 300 行。

model.py vs train.py 的分工

- train.py：训练样板（boilerplate）。我们有过的训练循环、AdamW、get_batch，它都有，只是复杂得多：保存/加载 checkpoint、学习率衰减、torch.compile 编译、分布式训练（多节点/多 GPU）。这些和模型本身无关，是"工程"。
- model.py：模型定义。和我们写的几乎一模一样——position/token embedding、Block、ln_f、lm_head、generate。

三个与我们不同的细节

1. Batched multi-head attention（4D 张量）

我们的写法是：多个 Head 各自算完再 cat。nanoGPT 用一个 `c_attn = nn.Linear(n_embd, 3 * n_embd)` 一次性算出所有头的 $q/k/v$ ，然后 `view + transpose` 成 4D 张量 (B, nh, T, hs) ——把"头"也当成一个 batch 维：

python

```
B=batch, T=time, C=n_embd, nh=n_head,
hs=head_size=n_embd//n_head
```

```
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
```

... 然后在 (B, nh, T, hs) 上做矩阵乘法 ...

```
y = y.transpose(1, 2).contiguous().view(B, T, C)
```

- 数学上和我们完全等价，只是把"头"塞进 batch 维、批量并行计算——效率更高，代码更紧凑。nanoGPT 还用了 Flash Attention（PyTorch 2.0 的 `scaled_dot_product_attention`，自带因果遮罩），让 GPU "brrrr"。

2. GeLU 非线性（替代 ReLU）

为什么用 GeLU 而不是 ReLU？一方面，nanoGPT 要能加载 GPT-2 预训练权重，必须对齐非线性；另一方面，GeLU 在实践中通常比 ReLU 效果稍好——它更平滑，负半轴不会完全"死掉"。这是工程对齐和性能优势的双赢。

python

```
self.gelu = nn.GELU()
```

- 为什么不用 ReLU？为了能加载 OpenAI 官方的 GPT-2 预训练权重。OpenAI 用了 GeLU，nanoGPT 要能 `from_pretrained('gpt2')` 加载 checkpoint，就必须对齐非线性。这是"工程妥协"的典型例子。

3. 参数分组：weight decay 与不 decay

python

2D 及以上（矩阵乘法的权重 + embedding）→ 施加 weight decay

```
decay_params = [p for n, p in param_dict.items() if p.dim() >= 2]
```

1D（bias、LayerNorm）→ 不施加

```
no_decay_params = [p for n, p in param_dict.items() if p.dim() < 2]
```

- 经验法则：权重矩阵 decay，偏置和归一化不 decay。这能提升泛化，也是训练大模型的标准做法。
- 另外还有：c_proj.weight 用 $0.02/\sqrt{2 \times n_{\text{layer}}}$ 缩放初始化（残差投影特殊初始化）、token embedding 与 lm_head 权重绑定（weight tying）等。细节更多，但骨架和你写的一模一样。

回到 ChatGPT / GPT-3：预训练 vs 微调

nanoGPT 专注的是预训练（pre-training）。想得到 ChatGPT，需要两个阶段。

阶段一：预训练（Pre-training）

这就是我们做的事——只是"婴儿版"：

- 在一大块互联网文本上训练一个 decoder-only Transformer，让它学会"补全文档"。
- 我们的对比：模型 ~10M 参数，数据 ~100 万字符（按 OpenAI 的 50K 子词词表折算大约 30 万 tokens）。
- GPT-3（2020 论文）：最大模型 175B 参数（是我们的约 1.75 万倍），训练于 300B tokens（比我们大约 100 万倍）。

	我们的 mini-GPT	GPT-3 最大
参数量	~10M	175B
训练 tokens	~300K	300B
架构	decoder-only Transformer	decoder-only Transformer（几乎相同）

- ^① 参数差约 4 个数量级、训练数据差约 6 个数量级，但架构几乎一样——这就是为什么"理解这 200 行代码"是有价值的。
- 预训练产出的不是助手，而是文档补全器：你问它问题，它可能回你更多问题、或者续写一篇新闻稿。行为完全不可控（"unhinged"）。它还不可用。

阶段二：微调 / 对齐（Fine-tuning / Alignment）

把"文档补全器"变成"问答助手"，大致三步（OpenAI 官方博客）：

文档补全器（预训练结果）

|

| ① Supervised Fine-Tuning (SFT)

▼

用"问题在上、答案在下"的问答格式数据微调

→ 学会"等待问题、给出答案"的格式

|

| ② 奖励模型（Reward Model）

▼

让模型对同一问题生成多个回答，人类标注排序

→ 训练一个独立的"奖励模型"预测哪个回答更受欢迎

|

| ③ RLHF / PPO

▼

用策略梯度（PPO）强化学习优化采样策略

→ 让生成的回答在奖励模型下期望得分最高

|



问答助手 (ChatGPT)

- SFT (监督微调)：用几千个"问答格式"的标注样例微调，让模型从"补全文档"转向"补全答案"。大规模模型的微调非常样本高效，几千条数据就能起作用。
- 奖励模型：人类对多个回答排序，训练一个模型预测"哪个回答更好"。
- RLHF/PPO：奖励模型当"评分器"，用强化学习 (策略梯度) 调整生成策略，让采样出的回答期望奖励最高。

> 一句总结：预训练 = 让模型学会"像文本一样说话"；微调 = 让模型学会"像助手一样回答"。前者的数据是海量互联网，后者的数据是人工标注的问答偏好，量级差了很远，且大多不公开。

总结与展望

这一路我们干了什么：

- 用约 200 行代码训练了一个 decoder-only Transformer (= GPT)
- 在 tiny Shakespeare 上从 bigram 的 ~2.5 一路降到 2.23 (CPU 缩小型)，GPU 完整版可达 1.48
- 生成的文本看起来像莎士比亚——虽然读起来无意义

这就是 ChatGPT 的骨架：预训练阶段与它同构；微调 (SFT/奖励模型/RLHF) 是加在它上面的"对齐"层。

- > 关于"Transformer 之后的路径"：我们的下一步可以是——
- > - 规模：用 GPU 跑完整超参 (1.48)，或读 nanoGPT 学分布式训练
 - > - 微调：学 SFT / LoRA / 奖励模型 / RLHF，把"补全器"变"助手"
 - > - 新架构：关注注意力之外的演进 (线性注意力、MoE、Mamba 等)
 - > - 推荐继续读 Karpathy 的 micrograd/minGPT/nanoGPT，它们是同一套思路的不同复杂度

如果还想深入，原视频在结尾建议："go forth and transform"。

学完本部分你能...

- 区分 decoder-only / encoder / encoder-decoder 三种 Transformer，画出翻译场景的 cross-attention 数据流
- 解释特殊 token (<START>/<END>) 在条件生成里的作用
- 读懂 nanoGPT 的 model.py：batched multi-head (4D)、GeLU、参数分组、Flash Attention
- 讲清 预训练 vs 微调：文档补全器 → (SFT → 奖励模型 → RLHF/PPO) → 问答助手
- 用参数量/tokens 把"我们的 10M / 30 万 tokens"和"GPT-3 的 175B / 300B tokens"对比
- 说出为什么"理解这 200 行代码"能迁移到理解 ChatGPT

课后练习

Q1: 为什么 GPT 用 decoder-only，而《Attention is All You Need》原文是 encoder-decoder？

A: 原文做机器翻译，需要 encoder 通读源语言 (法语)、再让 decoder

基于它生成目标语言 (英语)，所以有 encoder + cross-attention。GPT 是"文档补全器"：没有额外的条件输入，只需要自回归地生成文本——decoder-only (带三角遮罩) 恰好就是这种"无条件续写"所需的最小完整形态。任务决定了架构选择。

Q2: nanoGPT 的 batched multi-head 和我们的 ModuleList([Head(...)]) + cat 数学上等价吗？为什么 4D 实现更高效？

A: 完全等价。我们的写法是每个 Head 独立做 (B, T, hs) 的注意力的 $\text{wei} @ v$ ，再把结果在通道维 cat 起来；nanoGPT 用一个 Linear 一次算全部 $q/k/v$ ，view+transpose 成 (B, nh, T, hs) 把"头"当成 batch

维，一次批矩阵乘法并行算所有头。4D 版本把头的并行也纳入矩阵乘法（GPU 擅长的大块运算），且避免了逐个 Head 循环的调度开销，所以更高效。数学结果一模一样。

Q3: 为什么"预训练产出文档补全器，而不是助手"？微调阶段的三步各解决了什么问题？

A: 预训练的目标函数就是"预测下一个 token"（补全文档），所以模型只会补全——给它问题，它可能续写更多问题或新闻稿，行为不可控。微调三步逐步对齐：①SFT 用"问答格式"数据教它"等一个问题、答一个答案"的格式；②奖励模型把人类的"哪个回答更好"的偏好编码成一个可微的评分器；③RLHF/PPO 用强化学习让生成策略在奖励模型下期望得分最高。三步把"会说话"变成"会好好回答问题"。

完结

恭喜你完成 Part 6（Transformer / GPT）全部四章！

学完这套教程，去 Assignment 6 动手实践吧：

[Assignment 6](../../assignments/assignment_6/)

回顾完整路线：Part 1 Bigram → Part 2 MLP → Part 3 BatchNorm → Part 4 反向传播 → Part 5 WaveNet → Part 6 Transformer/GPT。现在你已经能从零构建一个语言模型家族了。

> 别忘了回到 README 的"演进路线"表格，对照一下每一步 loss 是怎么降下来的。

[← 上一章：Part 5 WaveNet](../../Part5_wavenet/tutorial/README.md)